



UNIVERSIDADE DO MINHO

Licenciatura em Engenharia Informática

Sistemas Distribuídos

2025/2026

Grupo 31

Base de dados para séries temporais

a106853, Ana Beatriz Freitas

a106793, Lucas André Dias Fernandes

a107367, João Azevedo

a106877, José Miguel Cação

Janeiro 2026

Índice

1	Introdução	2
2	Arquitetura do Sistema	2
2.1	Estrutura em Camadas	2
2.2	Protocolo de Comunicação e Gestão de Requisições	2
3	Concorrência e Sincronização	3
3.1	ThreadPool Personalizado	3
3.2	Estratégias de Sincronização e Primitivas	3
4	Sistema de Notificações Assíncronas	4
4.1	Arquitetura e Motivação	4
4.2	Coordenação com <i>Locks</i>	4
4.3	Execução Assíncrona e Submissão de <i>Callbacks</i>	4
4.4	Limpeza e Gestão de Ciclo de Vida	4
4.5	Deteção de Mudança de Dia	4
5	Persistência e Cache	4
5.1	Sistema de Persistência	4
5.2	Sistema de Cache Multi-Nível	5
6	Funcionalidades Especiais	5
7	Avaliação de Desempenho e Análise de Testes	5
7.1	Utilização de Ferramentas de IA na Criação dos Testes	5
7.2	Cenários de Teste	5
7.3	Resultados Obtidos	6
8	Conclusão	6
9	Anexos	7

1 Introdução

Este relatório documenta o desenvolvimento e a arquitetura de um sistema distribuído concebido para a gestão de séries temporais, implementado no âmbito da unidade curricular de Sistemas Distribuídos, da Licenciatura em Engenharia Informática na Universidade do Minho. A solução apresentada visa a resolução de desafios inerentes ao registo contínuo e processamento eficiente de grandes volumes de dados relacionados a eventos de vendas de produtos.

O objetivo principal do sistema é permitir a realização de consultas históricas otimizadas e agregações estatísticas sobre diferentes períodos. O sistema foi projetado para oferecer suporte escalável a operações concorrentes, garantindo consistência e alto desempenho, mesmo em cenários de carga elevada.

2 Arquitetura do Sistema

O sistema apresenta uma arquitetura cliente-servidor distribuída, projetada com base nos princípios de modularidade e separação de responsabilidades. Adota uma organização em camadas, cada uma com responsabilidades claramente delimitadas. O diagrama da Figura 1 ilustra a arquitetura geral do sistema para facilitar o entendimento.

2.1 Estrutura em Camadas

A arquitetura é composta por uma divisão lógica de responsabilidades entre o cliente e o servidor, com uma comunicação intermediada por *middleware* que assegura abstrações funcionais e baixo acoplamento entre os componentes.

Camadas no Cliente

No cliente, a camada de apresentação é representada pela *InterfaceUtilizador*, que interage diretamente com *stubs*. Estes encapsulam todos os detalhes da comunicação remota, fornecendo transparência em relação à localização dos serviços e ocultando a complexidade da serialização e envio. A camada de *ClientMiddleware* gere conexões remotas e utiliza o *Demultiplexer* para correlacionar requisições e respostas, permitindo o processamento simultâneo de múltiplos pedidos.

Além disso, o cliente suporta dois modos configuráveis:

- **Modo Dedicado:** Uma conexão persistente é mantida, ideal para casos de uso com carga contínua.
- **Modo Pool:** Conexões são obtidas de um *pool* à medida que são necessárias, otimizando o uso de recursos em cenários de requisições esporádicas.

Camadas no Servidor

No servidor, as conexões recebidas dos clientes são geridas por instâncias da classe *ClientHandler*, executadas num *ThreadPool*. As mensagens são encaminhadas ao *RequestDispatcher*, que identifica os *skeletons* apropriados. O *RequestDispatcher* também realiza validações e coleciona métricas de desempenho detalhadas, como latência e taxa de sucesso.

Os *skeletons*, são responsáveis por desserializar os *payloads*, invocar os serviços de domínio e serializar as respostas. Essa separação permite isolar a lógica do serviço dos detalhes do transporte de rede.

Persistência e Cache no Servidor

Os serviços utilizam *repositories* para a persistência, com interfaces como *IEventoRepository* e implementações como *EventoFileRepository*. A sincronização de acesso ao armazenamento é gerida por *ReadWriteLocks*, otimizando *workloads* predominantemente de leitura. Adicionalmente, um sistema de cache multi-nível, gerido pela *CacheManager*, melhora o desempenho em operações repetidas.

2.2 Protocolo de Comunicação e Gestão de Requisições

Comunicação entre cliente e servidor ocorre por *Data Transfer Objects*, reduzindo o tráfego na rede ao transportar apenas os dados necessários, protegendo a lógica interna do sistema.

O sistema utiliza um protocolo binário proprietário otimizado para reduzir a latência e maximizar a vazão (*throughput*). Cada mensagem inclui um prefixo com o tamanho em bytes, seguido pelo payload serializado via *ObjectOutputStream*. Existem dois formatos principais de mensagens:

- **Formato de Requisição:** [tag:long][serviceId:byte][methodId:byte][payload:Object]
- **Formato de Resposta:** [tag:long][payload:Object]

A classe `Message` unifica este formato, fornecendo (*factory methods*). O campo `tag` correlaciona requisições e respostas, permitindo a execução concorrente de múltiplas requisições. Os identificadores `serviceId` e `methodId`, substituem o uso reflexivo de métodos, reduzindo a sobrecarga associada.

Esta abordagem oferece controle total sobre o formato da mensagem, simplificando a possível aplicação de otimizações futuras, como compressão seletiva.

3 Concorrência e Sincronização

3.1 ThreadPool Personalizado

O sistema implementa o `ThreadPoolImpl`, uma solução personalizada que substitui os `Executors` da biblioteca padrão do Java, oferecendo controle granular sobre rejeições e finalizações, monitorização detalhada de *threads* ativas e tarefas pendentes e eficiência e robustez sob alta carga, com *threads* reutilizáveis.

A estrutura interna inclui uma fila de tarefas (`Queue<Runnable>`) e um conjunto de *threads* ativas (`Set<Thread>`), protegidos por `ReentrantLock`. A sincronização usa `Condition Variables` que sinalizam quando há tarefas disponíveis e coordenam o encerramento da *pool*.

Os *workers* ficam em um *loop* contínuo, aguardando por tarefas ou processando-as assim que disponibilizadas. Este *design* elimina o custo de criar e destruir *threads*.

O `ThreadPoolImpl` expande *threads* de forma dinâmica (*lazy initialization*) até o limite configurado, `maxThreads`. Quando a fila atinge a capacidade máxima, tarefas são rejeitadas de forma explícita, permitindo ao chamador tratar a rejeição adequadamente.

O *pool* suporta um *Shutdown* Gracioso, que conclui as tarefas pendentes antes de liberar recursos, e um *Forçado*, que interrompe imediatamente os processos e limpa a fila.

3.2 Estratégias de Sincronização e Primitivas

Uso de `ReadWriteLock`

Componentes como o `CacheManager` e os repositórios de dados, utilizam `ReentrantReadWriteLock` para otimizar acesso concorrente, permitindo múltiplas leituras concorrentes enquanto garante escrita isolada. No método `obterAgregacaoDia()`, o padrão é dividido em três etapas: busca na cache (`readLock`), cálculo da agregação (*lock-free*) e inserção na cache (`writeLock`).

Condition Variables e Coordenação

As `Condition Variables` são utilizadas para comunicação eficiente entre threads:

- **ThreadPool:** A `notEmpty` sinaliza a disponibilidade de tarefas na fila, acordando *workers* bloqueados.
- **Demultiplexer:** Threads aguardam respostas específicas por `tags` em condições distintas, despertadas seletivamente ao receber a mensagem correspondente.

Double-Checked Locking para Inicialização Tardia

O padrão *double-checked locking*, usado em instâncias como `ErrorLogger` e `RepositoryFactory`, permite inicialização *lazy* e eficiência no acesso simultâneo. Métodos de leitura concorrem sem bloqueios, enquanto a inicialização é protegida contra condições de corrida.

Agregações Paralelas e Otimizadas

A etapa de computação em `CacheManager.obterAgregacaoDia()` foi projetada para ser *lock-free*, calculando dados paralelamente e armazenando o primeiro resultado obtido. Este modelo evita contenção e reduz a latência em cenários de alta concorrência.

Streaming e Persistência

O `EventoFileRepository` usa `RandomAccessFile` para processar agregações em janelas temporais (*multi-dia*), lendo eventos sequencialmente sem sobrecarregar a memória. Esse *design* mantém eficiência com baixas exigências de memória, mesmo em grandes volumes de dados históricos.

4 Sistema de Notificações Assíncronas

4.1 Arquitetura e Motivação

O sistema inclui um `NotificationManager` que implementa uma arquitetura *Publisher/Subscriber* para notificações baseadas em eventos de vendas. Este serviço permite que os clientes registrem dinamicamente condições específicas em um ambiente distribuído, garantindo a execução assíncrona de *callbacks*.

A execução dos *callbacks* é realizada em um `ThreadPool` dedicado, `notificationPool`, separando as operações de notificação de outras camadas do sistema. Este *design* previne que *callbacks* potencialmente lentos afetem *threads* dedicadas ao processamento principal ou manipulação de eventos.

4.2 Coordenação com Locks

Para gerir notificações concorrentes, o `NotificationManager` emprega dois mapas principais para as condições de produtos específicos vendidos e vendas consecutivas de produtos no dia atual. As chaves nesses mapas seguem um formato consistente e normalizado para evitar conflitos, como `"min(p1,p2):max(p1,p2):dia"` para condições específicas. Este formato garante que diferentes ordens de produtos ((5,7) e (7,5)) correspondam à mesma chave. Todas as operações nestes mapas são protegidas por um `ReentrantLock`.

4.3 Execução Assíncrona e Submissão de Callbacks

Quando uma condição é satisfeita, o método `notificarHandlers()` processa os *handlers* associados à chave da condição. Cada *handler* é submetido ao `notificationPool`, que os executa de forma isolada em *threads* dedicadas:

- Se um *callback* lança uma exceção, esta é capturada e registrada através do `ErrorLogger`, sem afetar outros *callbacks*.
- Submits sucessivos garantem isolamento e previnem bloqueios no processamento principal do sistema.

Este *design* permite que o sistema lide com um grande número de clientes simultâneos enquanto mantém baixa latência nas notificações.

4.4 Limpeza e Gestão de Ciclo de Vida

Com base na rotação diária do sistema, o método `limparNotificacoesDia()` é invocado. Esta rotina remove todas as entradas de notificações associadas ao dia anterior, emite um `signalAll()` para acordar todas as *threads* que aguardam notificações referentes ao dia anterior, e garante uma interface limpa para o novo ciclo de dia, prevenindo acumulação de chaves obsoletas.

4.5 Detecção de Mudança de Dia

Para evitar espera indefinida, os clientes que invocam `notificarVendaEspecifica()` bloqueiam em condições sincronizadas. Durante este bloqueio, o campo `diaAtual` é consultado periodicamente para verificar possíveis avanços de dia.

Caso uma mudança de dia seja confirmada a função retorna imediatamente com `RespostaDTO.erro("Dia avançou")`, garantindo uma resposta consistente ao cliente. A mudança de dia no servidor limpa eventos antigos e atualiza o campo `diaAtual`.

5 Persistência e Cache

5.1 Sistema de Persistência

O sistema usa um protocolo de serialização binária para armazenar dados de forma compacta e eficiente. Essa abordagem foi escolhida para atender aos requisitos de alto volume, como o registo de milhões de eventos por dia, mantendo operações de leitura e escrita rápidas.

Formato Binário Personalizado

Os dados persistidos seguem um formato binário otimizado para reduzir o uso de espaço e aumentar a velocidade de *parsing*. Os dados persistentes incluem informações vitais ao sistema:

- **Utilizadores:** Serialização armazena cada utilizador como `[n:int][username:UTF][hash:UTF]`, onde *n* é o

número de utilizadores. Todos os dados são carregados em tempo de iniciação.

- **Eventos:** Os eventos são armazenados por dia como `[numProdutos:int][produtoID:int,numEventos:int,[qtd:int, preço:double]...]`, onde os produtos são ordenados com base no `produtoID`, garantindo buscas otimizadas.

Padrão de Repositório

Para aceder os dados persistentes, foram implementados repositórios seguindo o padrão `Repository`, encapsulando a lógica de armazenamento e permitindo troca transparente para outros mecanismos de persistência no futuro. Exemplos incluem `EventoFileRepository` e `UsuarioFileRepository`. São usadas também operações com `RandomAccessFile`, assegurando eficiência mesmo para consultas envolvendo múltiplos dias.

Todos os métodos críticos nos repositórios são protegidos por `ReadWriteLock`, permitindo leitura paralela enquanto serializa operações de escrita, o que reflete a carga de trabalho *read-heavy* typical do sistema.

5.2 Sistema de Cache Multi-Nível

O `CacheManager` organiza os dados do sistema em dois níveis principais de cache: Cache de Agregações e Séries em Memória.

Para evitar o consumo excessivo de memória, o cache da aplicação segue uma política LRU. Assim, quando o limite configurado `seriesEmMemoria.size()` é atingido, a série mais antiga é removida automaticamente.

Agregação Multi-Dia

Para consultar dados agregados de vários dias, a aplicação tenta utilizar a cache para cada dia individual. Caso os dados desejados abrangem mais de 5 dias fora da cache (*diasSemCache* > 5), o sistema executa agregações diretamente do repositório em modo *streaming*. Essa combinação de cache e *streaming* atinge um equilíbrio entre desempenho na consulta e consumo de memória.

6 Funcionalidades Especiais

O sistema inclui as seguintes funcionalidades:

- **Graceful Shutdown** coordenado entre cliente e servidor, onde o `ClientShutdownHandler` detecta a `ShutdownMessage` e liberta recursos, enquanto o servidor executa uma sequência controlada de notificação, fecho de `sockets` e finalização de `ThreadPools`;
- **Sistema de Métricas** através do `PerformanceMetrics`, que monitoriza continuamente contadores (`totalRequests`, `cacheHits/misses`), latência (mínimo, máximo e médio) e `throughput` (`requests * 1000 / uptimeMs`);
- **Logging Estruturado** via `ErrorLogger`, que rastreia eventos críticos e transições de estado, facilitando depuração e auditoria do sistema.

7 Avaliação de Desempenho e Análise de Testes

7.1 Utilização de Ferramentas de IA na Criação dos Testes

Recorreu-se a ferramentas de Inteligência Artificial exclusivamente como apoio à criação de código de teste, nomeadamente para a criação de clientes de teste concorrentes e para a automatização da submissão repetida de pedidos ao servidor.

O processo seguido consistiu na definição manual dos objetivos de cada cenário de teste, a formulação de *prompts* para gerar código de teste alinhado com esses objetivos, a análise crítica e validação manual do código gerado e ajustes manuais sempre que necessário para garantir a correção e adequação dos testes.

Os *prompts* utilizados na geração dos testes, bem como a validação efetuada ao código gerado, encontram-se disponíveis no ficheiro `prompts.txt`.

7.2 Cenários de Teste

Testes com Diferentes Tipos de Operações

Foram definidos cenários com diferentes perfis de carga, incluindo testes dominados por registo de eventos, testes de consultas de agregações e testes mistos. O objetivo foi observar o impacto de cada tipo de operação no desempenho do sistema (tempo

total e taxa de sucesso) e na sua capacidade de processamento.

No *workload* misto concorrente, uma parte dos clientes executa operações de filtragem e notificação com menor frequência, de forma a não introduzir instabilidade excessiva e manter o teste reproduzível. Neste cenário, observou-se **100% de sucesso para 20 clientes**, com tempo total de execução de **76 ms**, indicando que, para este nível de concorrência, o sistema suporta adequadamente uma mistura de operações (escrita e leitura), mantendo elevada disponibilidade.

Testes de Escalabilidade

Para avaliar a escalabilidade, o número de clientes concorrentes foi progressivamente aumentado. Em cada configuração foram recolhidas métricas de:

- **tempo total do teste** (segundos);
- **taxa de sucesso** (percentagem de clientes que completaram todas as operações);
- **throughput** aproximado (operações por segundo).

A Tabela 1 resume os resultados obtidos.

Table 1: Resultados dos testes de escalabilidade

Clientes	Sucessos	Falhas	Tempo(s)	Throughput(ops/s)	Taxa(%)
10	10	0	0.21	579.71	100.0
25	25	0	0.08	3750.00	100.0
50	42	8	0.08	6222.22	84.0
100	81	19	0.14	6992.81	81.0

Os resultados mostram que, até 25 clientes concorrentes, o sistema mantém 100% de taxa de sucesso. A partir de 50 clientes observa-se uma degradação da taxa de sucesso (para 84%), sugerindo que sob maior concorrência alguns clientes não conseguem completar o conjunto de operações dentro das condições do teste (e.g., timeouts, contenção, saturação de recursos). Ainda assim, com 100 clientes o sistema manteve 81% de sucesso, evidenciando que não ocorre colapso total e que o servidor continua a processar pedidos de forma significativa.

Em termos de throughput, observa-se um aumento com a concorrência, atingindo aproximadamente 6993 ops/s para 100 clientes. Este comportamento é compatível com um sistema que consegue aumentar o paralelismo e o débito global, mas à custa de uma maior probabilidade de falhas em alguns clientes quando a carga se torna elevada.

Testes de Robustez

Foram realizados testes de robustez em que alguns clientes não consumiam as respostas enviadas pelo servidor, simulando falhas ou comportamentos inesperados. O objetivo foi verificar se o servidor continua a responder a clientes bem comportados; se não ocorrem bloqueios globais (deadlocks) devido a clientes “lentos” ou que não leem respostas; se o sistema mantém algum progresso, mesmo com degradação de desempenho.

Os resultados destes testes indicaram que o servidor consegue continuar a servir clientes ativos mesmo na presença de clientes que não consomem respostas, sugerindo que o modelo de comunicação e a gestão de recursos não induzem bloqueio completo do sistema neste cenário.

7.3 Resultados Obtidos

Os resultados obtidos nos diferentes cenários de teste evidenciam tendências claras relativamente ao impacto da carga e do tipo de operações no desempenho do sistema. Observou-se uma degradação progressiva do tempo de resposta à medida que o número de clientes concorrentes aumenta, particularmente em cenários com elevada frequência de operações de escrita.

Os testes de robustez demonstraram que o servidor mantém a capacidade de responder a clientes ativos mesmo na presença de clientes que não consomem respostas, validando a robustez do modelo de comunicação adotado.

8 Conclusão

O desenvolvimento do sistema distribuído para gestão de séries temporais destacou a importância da aplicação de padrões de projeto e técnicas de sincronização adequadas para resolução de problemas de escalabilidade e concorrência. A solução final atendeu aos objetivos propostos, proporcionando uma plataforma eficiente. Os testes realizados validaram a capacidade do sistema de suportar cenários de carga elevada enquanto mantém a consistência dos dados e o desempenho do processamento.

9 Anexos

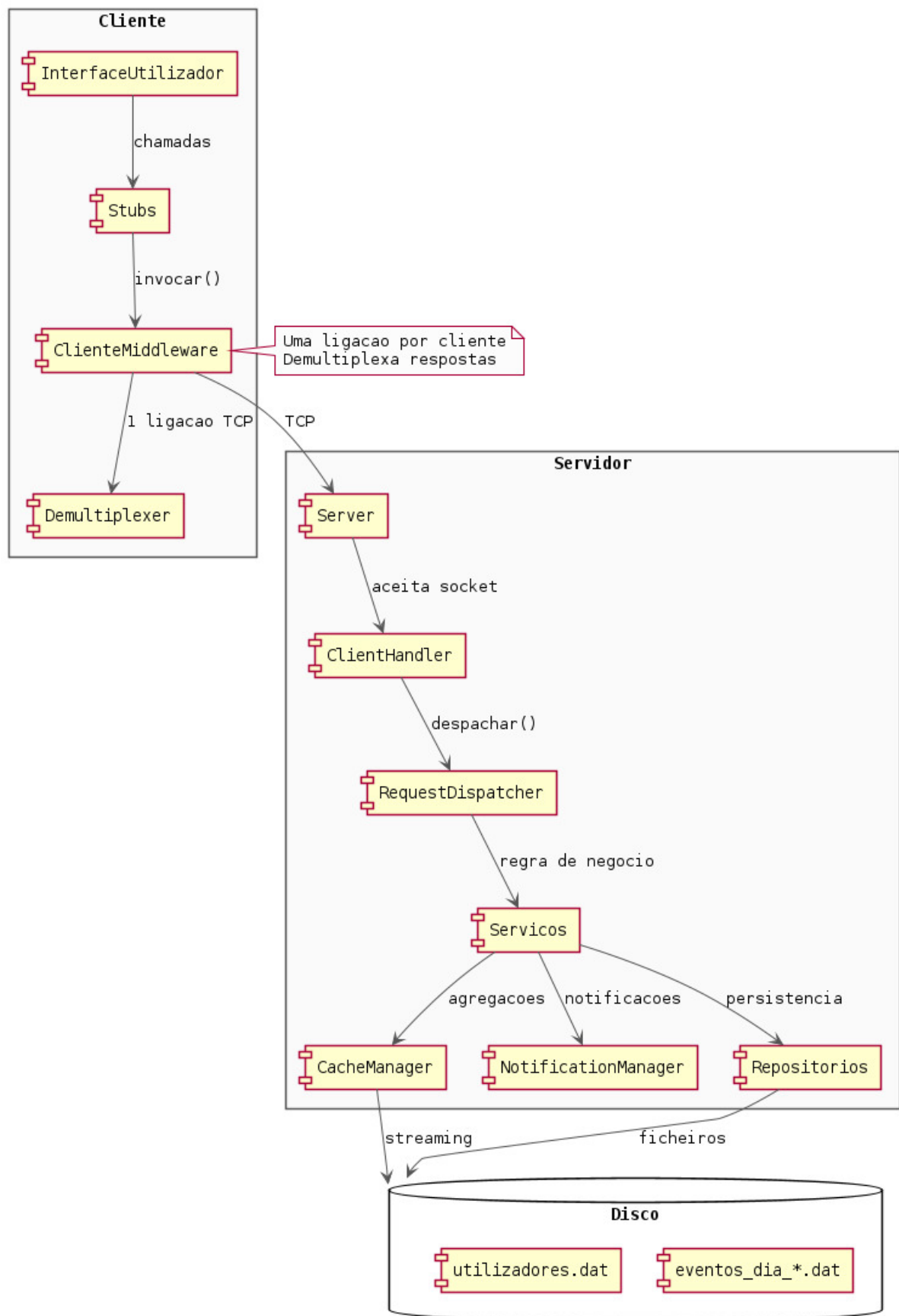


Figure 1: Diagrama detalhado da arquitetura do sistema, destacando os principais componentes do cliente e do servidor e sua comunicação.